

Comprendre notre projet d'intelligence artificielle, de A à Z

Fine-tuning d'un modèle de langage pour répondre à des questions de santé en 5 langues africaines — expliqué depuis zéro, pas à pas, notebook par notebook.

Adapté TDAH

Adapté dyslexie

Aucun pré-requis

Master 2 BI — UCAD

Projet : gemmafro-e2b — Multilingual Health QA

Dépôt : github.com/andilMc/gemmafro-e2b

Ce document explique **tout** le projet, du concept d'intelligence artificielle jusqu'au dernier fichier généré.

Comment lire ce document

Ce document est **long**, volontairement. Mais il est découpé en toutes petites parties pour que vous puissiez le lire par petits bouts, vous arrêter n'importe où, et reprendre plus tard sans être perdu.

REPÈRES VISUELS

Tout au long du document, vous croiserez toujours les mêmes 5 types de boîtes colorées. Une fois que vous les reconnaissez, votre œil sait quoi en faire sans avoir à tout relire :

DÉFINITION

Un mot compliqué expliqué simplement, la première fois qu'il apparaît.

POURQUOI C'EST IMPORTANT

Le lien entre une notion abstraite et une décision concrète du projet.

IMAGE POUR COMPRENDRE

Une comparaison avec quelque chose de la vie de tous les jours.

EXEMPLE CONCRET

Un vrai chiffre, une vraie phrase, tirés de notre projet.

CE QU'IL FAUT RETENIR

Le résumé en 2-3 lignes, à la fin de chaque partie. Si vous êtes pressé, lisez juste ces boîtes-là.

Conseils de lecture

- Ne lisez pas tout d'un coup. Une partie = une session de lecture. Il y en a 10.
- Chaque partie commence par une phrase noire en gras qui résume tout en une ligne — lisez-la d'abord.
- Le glossaire (fin du document) reprend **tous** les mots techniques, dans l'ordre alphabétique. Si un mot vous échappe en cours de route, allez-y directement.
- Ce n'est pas un examen. Il n'y a pas d'ordre obligatoire — vous pouvez sauter à la Partie 7 (les notebooks) si c'est ce qui vous intéresse le plus aujourd'hui.

Table des matières

Partie 1 — Les bases de l'intelligence artificielle (depuis zéro)	4
Partie 2 — Les modèles de langage (LLM)	7

Partie 3 — La problématique de notre projet	9
Partie 4 — Ce que l'analyse des données nous a appris	11
Partie 5 — Les solutions possibles, et pourquoi ce choix	13
Partie 6 — LoRA et QLoRA : les poids et les matrices, en détail	15
Partie 7 — Le voyage à travers chaque notebook	17
Partie 8 — Tous les fichiers générés, et à quoi ils servent	20
Partie 9 — Comprendre nos résultats	22
Partie 10 — Les problèmes techniques rencontrés, expliqués simplement	24
Partie 11 — Glossaire complet	26
Partie 12 — Pour aller plus loin	28

Les bases de l'intelligence artificielle (depuis zéro)

En une phrase : une IA, c'est un programme qui apprend à faire une tâche en regardant énormément d'exemples, au lieu qu'un humain lui dise exactement quoi faire ligne par ligne.

1.1 — C'est quoi l'intelligence artificielle ?

Un programme informatique classique, c'est une suite de règles écrites par un humain : "si X, alors fais Y". Une calculatrice ne devine rien, elle applique une règle fixe.

L'intelligence artificielle (IA), c'est différent. Au lieu d'écrire toutes les règles à la main, on donne au programme des **milliers d'exemples**, et c'est le programme lui-même qui trouve les règles, petit à petit, en s'ajustant à chaque erreur.

IMAGE POUR COMPRENDRE

Apprendre à un enfant à reconnaître un chat : vous ne lui donnez pas une liste de règles ("un chat a 4 pattes, des moustaches, ..."). Vous lui montrez plein de photos en disant "ça, c'est un chat" / "ça, ce n'est pas un chat". Après des centaines d'exemples, l'enfant reconnaît un chat qu'il n'a jamais vu. L'IA fonctionne pareil, mais avec des millions d'exemples et des calculs à la place d'un cerveau.

1.2 — C'est quoi le Machine Learning ?

DÉFINITION

Le **Machine Learning** (apprentissage automatique) est la branche de l'IA où le programme apprend à partir de données, sans qu'on lui dise explicitement la règle à suivre.

Notre projet est un projet de Machine Learning : on ne code pas nous-même "si la question contient le mot X, réponds Y". On donne au modèle 29 815 exemples de questions/réponses, et il apprend le lien entre les deux tout seul.

1.3 — C'est quoi un réseau de neurones ?

DÉFINITION

Un **réseau de neurones** est un programme organisé en couches de petites unités de calcul ("neurones artificiels"), inspiré (de très loin) du cerveau humain. Chaque neurone reçoit des nombres, fait un petit calcul, et transmet le résultat au neurone suivant.

Un grand modèle de langage (comme celui qu'on utilise, Gemma) est un immense réseau de neurones, organisé en dizaines de couches empilées, avec des milliards de connexions entre les neurones.

1.4 — C'est quoi un "poids" (weight) ?

IMAGE POUR COMPRENDRE

Imaginez une table de mixage de studio de musique, avec des milliers de petits boutons (potentiomètres). Chaque bouton contrôle "combien" un signal compte dans le résultat final. Un **poids**, c'est exactement ça : un petit nombre qui dit "à quel point cette information compte" dans le calcul d'un neurone.

Un modèle comme Gemma a des **milliards** de ces petits boutons (poids). Au départ (avant tout entraînement), ils sont réglés au hasard. **Entraîner** un modèle, c'est tourner tous ces boutons, un tout petit peu à chaque fois, jusqu'à ce que le résultat final soit bon.

1.5 — C'est quoi une matrice ?

DÉFINITION

Une **matrice**, c'est simplement un grand tableau de nombres, organisé en lignes et en colonnes. Rien de plus qu'un tableau Excel rempli de chiffres.

Dans un réseau de neurones, les poids ne sont pas rangés en vrac : ils sont organisés en matrices, parce que ça permet de faire les calculs très rapidement (les ordinateurs, et surtout les cartes graphiques/GPU, sont excellents pour multiplier des matrices entre elles).

IMAGE POUR COMPRENDRE

Reprenez l'image de la table de mixage : au lieu d'avoir les boutons éparpillés n'importe comment, ils sont rangés proprement en grille, comme les cases d'un plateau de jeu — ligne 1, colonne 1 ; ligne 1, colonne 2 ; etc. C'est ça, une matrice de poids.

1.6 — C'est quoi "entraîner" un modèle ?

L'entraînement, c'est une boucle répétée des millions de fois :

- 1 Le modèle regarde un exemple (une question) et fait une prédiction (une réponse) avec ses poids actuels.
- 2 On compare sa prédiction à la vraie réponse attendue. Plus c'est différent, plus l'**erreur** (on dit aussi la **perte**, ou *loss* en anglais) est grande.
- 3 Un calcul mathématique (la **rétropropagation du gradient**) détermine, pour chaque poids, dans quel sens et de combien il faut le tourner pour réduire un peu cette erreur.
- 4 On tourne tous les poids concernés d'un tout petit cran dans la bonne direction.
- 5 On recommence avec l'exemple suivant, encore, et encore.

DÉFINITION

Une **époque** (*epoch*), c'est le fait d'avoir fait passer **tous** les exemples d'entraînement une fois dans cette boucle. Notre modèle a fait 3 époques : il a vu chaque exemple 3 fois.

CE QU'IL FAUT RETENIR

Une IA apprend par l'exemple plutôt que par des règles écrites à la main. Le "savoir" du modèle est stocké dans des millions/milliards de petits nombres appelés **poids**, rangés en grands tableaux appelés **matrices**. Entraîner le modèle, c'est ajuster ces poids petit à petit pour réduire ses erreurs.

Les modèles de langage (LLM)

En une phrase : un LLM est un immense réseau de neurones entraîné à faire une seule chose extrêmement bien — deviner quel mot vient après les mots précédents.

2.1 — C'est quoi un LLM ?

DÉFINITION

LLM = *Large Language Model*, "grand modèle de langage". C'est un réseau de neurones entraîné sur d'énormes quantités de texte (livres, sites web, articles...), pour apprendre à produire du texte qui a du sens.

Gemma (le modèle qu'on utilise, publié par Google) est un LLM. ChatGPT, Claude (moi-même), Gemini... sont aussi des LLM.

2.2 — La tokenisation : couper le texte en petits morceaux

DÉFINITION

Un ordinateur ne comprend que des nombres, jamais du texte directement. La **tokenisation**, c'est l'étape qui découpe une phrase en petits morceaux (des **tokens** : un mot entier, ou parfois juste un bout de mot), pour ensuite transformer chaque morceau en nombre.

IMAGE POUR COMPRENDRE

C'est comme découper une phrase en pièces de puzzle. "Bonjour le monde" pourrait devenir 3 pièces : "Bonjour" / " le" / " monde". Chaque pièce a un numéro dans un immense dictionnaire (le **tokenizer**).

POURQUOI C'EST IMPORTANT POUR NOTRE PROJET

En analysant nos données (Partie 4), on a découvert que le découpeur (tokenizer) de Gemma découpe le luganda et l'amharique en **beaucoup plus** de petits morceaux que l'anglais pour dire la même chose (3 tokens par mot en moyenne en luganda, contre 1,25 en anglais). Ça veut dire que le modèle "connaît" moins bien ces langues, et que ça lui coûte plus cher en calcul de les traiter.

2.3 — Les embeddings : transformer les mots en listes de nombres

DÉFINITION

Un **embedding**, c'est une liste de plusieurs centaines de nombres qui représente le "sens" d'un token. Deux mots avec un sens proche (ex. "médecin" et "docteur") ont des listes de nombres proches.

Cette transformation se fait dans une immense matrice appelée la **table d'embeddings** (ou *embedding table*) : une ligne par token possible dans le vocabulaire du modèle, et chaque ligne contient les nombres qui représentent ce token. Chez Gemma, cette table est très grande car le vocabulaire couvre énormément de langues.

2.4 — Comment un LLM génère du texte

Un LLM ne "sait" pas la réponse d'un coup. Il génère un token à la fois :

- 1 Il regarde tout le texte déjà écrit (la question + le début de la réponse).
- 2 Il calcule, pour chaque token possible de son vocabulaire, la probabilité que ce soit le prochain.
- 3 Il choisit le token le plus probable (c'est ce qu'on appelle la génération "gloutonne"/*greedy*, la méthode qu'on a utilisée pour l'évaluation).
- 4 Il ajoute ce token au texte, et recommence à l'étape 1 — jusqu'à un signal de fin, ou une limite de longueur.

IMAGE POUR COMPRENDRE

C'est le principe du "texte prédictif" de votre téléphone quand vous écrivez un SMS, mais démultiplié : au lieu de proposer 3 mots suivants approximatifs, le LLM a été entraîné sur des milliards de phrases et peut enchaîner des réponses longues, cohérentes et adaptées au contexte.

2.5 — Pré-entraînement vs fine-tuning

DÉFINITION

Le **pré-entraînement** est la phase (très longue, très coûteuse, faite par Google pour Gemma) où le modèle apprend le langage en général à partir d'un immense volume de texte. Le **fine-tuning** ("ajustement fin") est une phase courte, faite ensuite par nous, qui spécialise un modèle déjà pré-entraîné sur une tâche précise, avec un jeu de données beaucoup plus petit.

POURQUOI C'EST IMPORTANT POUR NOTRE PROJET

On n'a jamais entraîné un modèle "depuis zéro" — ça demanderait des millions d'euros de calcul. On est partis d'un modèle déjà pré-entraîné par Google (`gemma-4-E2B-it`) qui sait déjà parler plusieurs langues, et on l'a juste **spécialisé** sur les questions-réponses de santé de notre dataset. C'est tout l'objet de ce projet.

CE QU'IL FAUT RETENIR

Un LLM transforme le texte en tokens, puis en nombres (embeddings), et génère sa réponse un mot à la fois en devinant le plus probable. Un LLM pré-entraîné sait déjà "parler" en général ; le fine-tuning le spécialise sur une tâche précise, sans repartir de zéro.

La problématique de notre projet

En une phrase : des millions de personnes en Afrique n'ont pas accès à une information de santé fiable, parce qu'elle n'existe presque jamais dans leur langue maternelle.

3.1 — Le vrai problème derrière ce projet

Ce projet n'est pas un exercice technique gratuit. Il répond à un vrai constat, documenté par une organisation appelée **HASH** :

- En Afrique subsaharienne, le taux de maternité précoce est très élevé : 93 naissances pour 1 000 filles de 15 à 19 ans en 2023.
- Plus d'un million d'infections sexuellement transmissibles (IST) sont contractées chaque jour dans le monde, la majorité en Afrique.
- L'information de santé numérique disponible existe presque uniquement en anglais ou en français — pas dans les langues que parlent réellement les gens au quotidien (akan, luganda, swahili, amharique...).

3.2 — Qui organise ce défi ?

DÉFINITION

HASH (*Hub for Artificial Intelligence in Maternal, Sexual and Reproductive Health*) est un consortium ougandais (Infectious Diseases Institute, Sunbird AI, Makerere University) qui finance et organise des projets d'IA responsable pour la santé maternelle, sexuelle et reproductive en Afrique. C'est HASH qui a lancé, avec la plateforme **Zindi** (une plateforme de compétitions de data science africaine) et **l'ITU**, le défi dont on utilise le jeu de données.

PRÉCISION IMPORTANTE

Ce projet est un **exercice académique** : on utilise les données du défi Zindi, mais on ne soumet rien officiellement à la compétition. L'objectif est pédagogique — apprendre en construisant un vrai pipeline d'IA sur un vrai problème.

3.3 — La tâche exacte à résoudre

On donne au modèle une question de santé, écrite dans une langue donnée. Le modèle doit répondre, dans **la même langue**, de façon claire et fiable médicalement.

EXEMPLE CONCRET

Question (en amharique) : "Si une femme enceinte a la syphilis, en combien de temps doit-elle être traitée ?"
Réponse attendue (en amharique) : "Immédiatement. Pour éviter que l'infection ne se transmette au fœtus, le traitement à la pénicilline doit être administré le plus vite possible après le diagnostic."

3.4 — Les 5 langues et les données

Le jeu de données couvre 5 langues, réparties sur 8 combinaisons langue-pays (car "anglais" apparaît dans 4 pays différents, avec des questions/réponses propres à chaque contexte local) :

Code	Langue	Pays
Eng	Anglais	Ghana, Ouganda, Kenya, Éthiopie
Aka	Akan	Ghana
Lug	Luganda	Ouganda
Swa	Swahili	Kenya
Amh	Amharique	Éthiopie

Trois fichiers nous ont été fournis :

Fichier	Rôle	Nombre de lignes
Train.csv	Sert à entraîner le modèle	29 815
Val.csv	Sert à vérifier les progrès, sans jamais s'en servir pour entraîner	6 686
Test.csv	Questions seules, à qui on doit fournir des réponses	2 618

CE QU'IL FAUT RETENIR

Le problème est réel : l'accès à l'information santé dans les langues locales africaines. La tâche est simple à formuler (question dans une langue → réponse dans la même langue), même si elle est difficile à résoudre techniquement. On dispose de 29 815 exemples pour apprendre au modèle, et 2 618 questions "d'examen final" sur lesquelles il n'a jamais vu la réponse.

Ce que l'analyse des données nous a appris (EDA)

En une phrase : avant de construire quoi que ce soit, on a regardé les données en détail — et ce qu'on a trouvé a directement influencé toutes les décisions techniques qui ont suivi.

4.1 — Pourquoi regarder les données avant de coder

DÉFINITION

L'**EDA** (*Exploratory Data Analysis*, analyse exploratoire des données) est l'étape où on inspecte un jeu de données — tailles, distributions, valeurs manquantes, exemples — **avant** de construire un modèle dessus.

POURQUOI C'EST IMPORTANT

Un modèle entraîné sans avoir compris ses données, c'est comme cuisiner sans avoir goûté les ingrédients. On risque de rater des problèmes évidents (déséquilibres, biais) qui expliqueraient ensuite des résultats bizarres.

4.2 — Premier constat : les données sont propres

Aucune valeur manquante, aucun identifiant (`ID`) dupliqué, dans aucun des 3 fichiers. Un bon point de départ.

4.3 — Deuxième constat : les langues ne sont pas équilibrées

Certaines langues ont beaucoup plus d'exemples que d'autres dans `Train.csv` :

Langue	Nombre de lignes	Part du total
Eng_Uga (anglais, Ouganda)	7 624	25,6 %
Aka_Gha (akan)	4 455	14,9 %
Eng_Gha (anglais, Ghana)	4 443	14,9 %
Eng_Eth (anglais, Éthiopie)	3 915	13,1 %
Lug_Uga (luganda)	3 383	11,3 %
Eng_Ken (anglais, Kenya)	2 080	7,0 %
Swa_Ken (swahili)	2 070	6,9 %
Amh_Eth (amharique)	1 845	6,2 %

IMAGE POUR COMPRENDRE

Imaginez apprendre à un enfant à reconnaître des animaux avec 100 photos de chats, mais seulement 16 photos de zèbres. Il sera bien meilleur avec les chats. C'est le même risque ici : sans correction, le modèle risquait de bien mieux répondre en anglais qu'en amharique, simplement parce qu'il aurait vu 4 fois moins d'exemples d'amharique.

4.4 — Troisième constat : certaines langues sont "plus difficiles" pour le modèle

On a mesuré combien de **tokens** (morceaux de mots, cf. Partie 2) le découpeur de Gemma produit par mot, dans chaque langue :

Langue	Tokens par mot (moyenne)
Luganda	3,06
Amharique	3,01
Akan	2,24
Swahili	2,09
Anglais (moyenne des 4 variantes)	~1,28

POURQUOI C'EST IMPORTANT

Un ratio élevé veut dire que le découpeur "connaît mal" cette langue : il doit la couper en tout petits morceaux au lieu de mots entiers. C'est un signe que Gemma a été pré-entraîné avec très peu de texte en luganda/amharique. Résultat concret plus loin (Partie 9) : ce sont exactement ces langues qui obtiennent les scores les plus faibles, même après fine-tuning.

On a aussi vu que les réponses en akan et en luganda sont naturellement beaucoup plus longues (233 à 245 tokens en moyenne, jusqu'à ~1000 tokens pour certaines) que les réponses en anglais (32 à 121 tokens en moyenne).

4.5 — Quatrième constat : beaucoup de réponses se répètent

39,4 % des réponses de `Train.csv` sont des doublons exacts d'une autre réponse. Ce n'est pas une erreur : le jeu de données est de type "FAQ santé", où plusieurs questions différentes mais proches partagent souvent la même réponse officielle.

CE QU'IL FAUT RETENIR

L'EDA a révélé 3 problèmes à anticiper : (1) un déséquilibre fort entre langues (facteur ~4), (2) une difficulté intrinsèque plus grande pour akan/luganda/amharique (tokenisation), (3) beaucoup de réponses répétées. Ces 3 constats ont directement guidé les choix de la Partie 6 (comment on a préparé les données pour l'entraînement).

Les solutions possibles, et pourquoi ce choix

En une phrase : il existait 3 grandes façons de résoudre ce problème — on a choisi la plus ambitieuse, le fine-tuning d'un LLM, parce qu'elle permet les réponses les plus naturelles.

5.1 — Solution A : chercher la réponse la plus proche (TF-IDF)

DÉFINITION

Le **TF-IDF** (*Term Frequency - Inverse Document Frequency*) est une technique qui compare des textes en comptant les mots qu'ils ont en commun (en donnant plus de poids aux mots rares, plus informatifs que les mots courants comme "le", "de", "et").

La méthode : pour une nouvelle question, on cherche dans `Train.csv` la question la plus "ressemblante" (au sens du TF-IDF), et on recopie **telle quelle** la réponse associée.

IMAGE POUR COMPRENDRE

C'est comme chercher dans un vieux classeur de fiches la fiche qui ressemble le plus à votre question, et recopier ce qui est écrit dessus — sans jamais rien inventer ni reformuler.

Limite : si aucune question du jeu d'entraînement ne ressemble vraiment à la nouvelle question, la réponse recopiée sera hors sujet. Aucune génération, aucune créativité, aucune capacité à reformuler.

5.2 — Solution B : un modèle traducteur/générateur classique (mT5)

DÉFINITION

mT5 est un modèle multilingue de type *seq2seq* (séquence-à-séquence) : il prend un texte en entrée et génère un texte en sortie, entraîné spécifiquement pour ce genre de transformation (comme la traduction).

Contrairement au TF-IDF, mT5 génère vraiment une nouvelle réponse — mais il n'est pas "instruction-tuned" (pas spécialement entraîné à suivre des instructions comme un assistant conversationnel), et sa version la plus légère a une compréhension du langage moins fine qu'un LLM récent comme Gemma.

5.3 — Solution C (notre choix) : fine-tuner un LLM instruction-tuned

On a choisi de partir d'un LLM déjà entraîné à suivre des instructions et à converser (`google/gemma-4-12B-it` , le "it" signifiant *instruction-tuned*), et de le **spécialiser** sur notre tâche via le fine-tuning (cf. Partie 2.5 et Partie 6).

5.4 — Tableau comparatif

Critère	TF-IDF (retrieval)	mT5 (seq2seq)	Fine-tuning LLM (notre choix)
Génère du texte nouveau ?	Non — recopie	Oui	Oui
Qualité de langage attendue	Faible	Moyenne	Meilleure
Vitesse	Très rapide	Moyenne	Plus lente
Faisable sur un GPU gratuit ?	Oui, facilement	Oui	Oui, grâce à la quantification 4-bit (Partie 6)

POURQUOI CE CHOIX, CONCRÈTEMENT

On voulait des réponses fluides et bien formulées, pas de simples copier-coller de réponses existantes. Le fine-tuning d'un LLM récent est aussi l'approche la plus représentative des pratiques actuelles en IA appliquée — un choix pédagogiquement plus riche pour ce projet de classe.

CE QU'IL FAUT RETENIR

Trois approches étaient possibles : copier une réponse existante (TF-IDF), générer avec un modèle classique (mT5), ou spécialiser un grand modèle de langage récent (fine-tuning LLM). On a choisi la troisième pour la qualité de génération attendue et sa pertinence pédagogique.

LoRA et QLoRA : les poids et les matrices, en détail

En une phrase : au lieu de re-régler les milliards de boutons du modèle entier, on ajoute un tout petit panneau de réglage à côté, et on n'ajuste que celui-là.

6.1 — Pourquoi on ne peut pas juste "réentraîner tout Gemma"

Le modèle `gemma-4-E2B-it` contient des milliards de poids (Partie 1.4). Réentraîner **tous** ces poids ("fine-tuning complet") demanderait :

- Une quantité de mémoire GPU énorme (bien plus que les 16 à 24 Go dont on disposait sur Google Colab).
- Un temps de calcul très long.
- Un risque d'"oublier" les capacités générales du modèle en le sur-spécialisant sur seulement 53 300 exemples.

6.2 — Le principe de LoRA

DÉFINITION

LoRA (*Low-Rank Adaptation*) est une technique qui **gèle** tous les poids originaux du modèle (ils ne changent plus jamais) et ajoute, à côté de certaines matrices existantes, deux toutes petites matrices supplémentaires qu'on entraîne à la place.

IMAGE POUR COMPRENDRE

Reprenez l'image de la table de mixage géante (Partie 1.4). LoRA, c'est comme visser un petit boîtier de correction juste à côté, avec seulement quelques boutons. On ne touche à aucun bouton de la table originale — on ajoute juste une petite correction, calculée pour orienter le résultat final dans la bonne direction. À la fin, seul ce petit boîtier a été réglé, pas toute la table.

Concrètement, pour chaque grande matrice de poids ciblée (nos "projections d'attention" : `q_proj`, `k_proj`, `v_proj`, `o_proj`, et nos "projections du réseau" : `gate_proj`, `up_proj`, `down_proj`), LoRA ajoute deux petites matrices A et B. Le résultat final utilisé par le modèle est : *poids original (gelé) + (A × B)*.

DÉFINITION — LE "RANG" (R)

Le **rang** (paramètre `r`) contrôle la taille de ces petites matrices A et B — plus `r` est grand, plus elles peuvent apprendre de choses fines, mais plus elles prennent de mémoire et de temps à entraîner. On a choisi `r = 16`, une valeur courante et raisonnable pour ce genre de tâche.

POURQUOI C'EST IMPORTANT

Résultat mesuré dans notre projet : l'adaptateur LoRA final ne pèse que **~115 Mo**, alors que le modèle complet en pèse plusieurs milliards de fois plus de paramètres. On entraîne et on stocke une toute petite fraction du modèle, pour un résultat proche d'un fine-tuning complet.

6.3 — Le principe de QLoRA (la version "économe")

DÉFINITION

QLoRA = **Q**uantized LoRA. On ajoute une étape supplémentaire : le modèle de base gelé est chargé en mémoire dans un format **compressé** (4 bits par nombre au lieu de 16 ou 32), ce qui divise sa taille en mémoire par environ 4.

IMAGE POUR COMPRENDRE

C'est comme compresser une photo en JPEG plutôt que de garder le fichier brut de l'appareil photo : le fichier prend beaucoup moins de place, la qualité perd un tout petit peu de précision, mais reste largement suffisante pour l'usage qu'on en fait.

Grâce à QLoRA, notre modèle (plusieurs milliards de paramètres au total) tenait dans seulement **~6,7 Go** de mémoire GPU une fois chargé en 4-bit — ce qui rentre confortablement dans les 16 à 24 Go dont on disposait sur Google Colab.

6.4 — Nos réglages, expliqués simplement

Réglage	Valeur	Ce que ça veut dire
Quantification	4-bit (NF4)	Le modèle gelé est compressé pour tenir en mémoire
Rang (<code>r</code>)	16	Taille des petites matrices ajoutées par LoRA
Alpha (<code>lora_alpha</code>)	32	Un multiplicateur qui règle "l'intensité" de la correction LoRA (usuellement $2\times r$)
Dropout	0,05	Une petite dose de "hasard" pendant l'entraînement pour éviter que le modèle n'apprenne les exemples par cœur
Modules ciblés	7 types de couches	Les projections d'attention et de MLP (les parties du modèle qui traitent le contexte et la logique)

CE QU'IL FAUT RETENIR

LoRA gèle le modèle géant et n'entraîne que de petites matrices additionnelles insérées dans certaines couches — beaucoup plus léger qu'un fine-tuning complet. QLoRA ajoute la compression 4-bit du modèle gelé pour qu'il tienne en mémoire. Résultat : un modèle de plusieurs milliards de paramètres, entraînable sur un GPU accessible, avec un "supplément" appris de seulement **~115 Mo**.

Le voyage à travers chaque notebook

En une phrase : le projet est découpé en 6 notebooks, comme les 6 étapes d'une recette de cuisine, chacun préparant le terrain pour le suivant.

IMAGE POUR COMPRENDRE : LA RECETTE DE CUISINE

00 = installer la cuisine (les outils). **01** = inspecter les ingrédients. **02** = préparer/découper les ingrédients. **03** = cuisiner (l'entraînement, la partie la plus longue). **04** = goûter le plat avant de servir (évaluation). **05** = dresser l'assiette et servir (prédictions finales).

7.0 — Notebook 00 : installer l'atelier (Phase 0)

Objectif : préparer l'environnement de travail sur Google Colab, avant de toucher au modèle.

Algorithme, étape par étape :

- 1 Vérifier qu'un GPU est bien attribué par Colab (commande `nvidia-smi`).
- 2 Connecter Google Drive, pour pouvoir sauvegarder des fichiers qui survivent à la fermeture de la session (sinon, tout est perdu à la déconnexion).
- 3 Cloner (télécharger) le dépôt GitHub du projet, pour récupérer les données et le code.
- 4 Installer les bibliothèques Python nécessaires (`transformers` , `peft` , `bitsandbytes` ...).
- 5 Se connecter à Hugging Face (la plateforme qui héberge le modèle Gemma) avec un token d'accès.
- 6 Charger le modèle une première fois, en test, pour vérifier que tout fonctionne avant d'aller plus loin.

7.1 — Notebook 01 : explorer les données (Phase 1, EDA)

Objectif : comprendre le jeu de données avant d'y toucher (détails en Partie 4).

Algorithme : charger les 3 fichiers CSV → vérifier valeurs manquantes/doublons → compter les lignes par langue → mesurer la longueur des textes en tokens par langue → calculer le ratio tokens/mot par langue → repérer les réponses dupliquées → tout afficher sous forme de tableaux et de graphiques.

7.2 — Notebook 02 : préparer les données (Phase 2)

Objectif : transformer les questions/réponses brutes en un format que le modèle peut apprendre, en corrigeant les problèmes trouvés en Partie 4.

Algorithme :

- 1 Pour chaque exemple, construire une instruction ("Réponds à cette question en [langue]...") avec la vraie question.

- 2 Mettre en forme au format "conversation" attendu par Gemma (tour `user`, tour `model`).
- 3 Séparer 95 % des données pour l'entraînement, 5 % pour surveiller les progrès en cours de route (sans jamais s'en servir pour entraîner).
- 4 Dupliquer un peu plus les exemples des langues minoritaires (amharique, swahili, kenya anglais...), pour réduire le déséquilibre repéré en Partie 4.3 — sans dupliquer à l'infini, pour éviter que le modèle apprenne les phrases par cœur.
- 5 Mesurer la vraie longueur (en tokens) des exemples, pour choisir une limite de longueur adaptée (512 tokens).
- 6 Sauvegarder le résultat sur Google Drive, prêt à être réutilisé par le notebook suivant.

7.3 — Notebook 03 : le fine-tuning (Phase 3 — la cuisine)

Objectif : entraîner réellement le modèle. C'est le notebook le plus long à exécuter (plusieurs heures).

Algorithme :

- 1 Charger le modèle de base en 4-bit (QLoRA, cf. Partie 6.3).
- 2 Geler tous ses poids, puis ajouter les petites matrices LoRA (cf. Partie 6.2) sur les couches ciblées.
- 3 Lancer la boucle d'entraînement (cf. Partie 1.6) : lire un petit paquet d'exemples (un "batch"), calculer l'erreur, ajuster les petites matrices LoRA, répéter — pendant 3 passages complets sur les 53 300 exemples (3 "époques").
- 4 Sauvegarder régulièrement l'état d'avancement ("checkpoint") sur Drive, pour pouvoir reprendre si la session Colab se coupe.
- 5 À la fin, sauvegarder l'adaptateur LoRA final (les petites matrices entraînées, ~115 Mo).

EXEMPLE CONCRET — LE RÉSULTAT DE CET ENTRAÎNEMENT

9 996 pas d'entraînement, sur 3 époques, pour une **perte finale de 0,668** (plus ce nombre est bas, moins le modèle se trompe — cf. Partie 1.6). Temps de calcul cumulé : environ 16,8 heures, réparties sur plusieurs sessions Colab.

7.4 — Notebook 04 : goûter le résultat (Phase 4, évaluation)

Objectif : mesurer si le fine-tuning a vraiment aidé, avec des chiffres et des exemples.

Algorithme :

- 1 Recharger le modèle de base + l'adaptateur LoRA entraîné.
- 2 Prendre 500 questions de `Val.csv` (jamais vues à l'entraînement).
- 3 Générer une réponse pour chacune, **avec** l'adaptateur activé (modèle fine-tuné).
- 4 Générer une réponse pour les mêmes questions, **sans** l'adaptateur (modèle "zero-shot" — le modèle Google original, tel quel).

- 5 Comparer chaque réponse générée à la vraie réponse attendue, avec un score appelé ROUGE (cf. Partie 9).
- 6 Calculer ces scores globalement et langue par langue, pour voir où le modèle est fort ou faible.

7.5 — Notebook 05 : servir le plat (Phase 6, prédictions finales)

Objectif : produire les réponses finales sur les 2 618 questions de `Test.csv` — celles pour lesquelles on n'a jamais eu la réponse.

Algorithme : même principe que le notebook 04 (charger le modèle + l'adaptateur, générer les réponses), mais sur tout `Test.csv`, avec des vérifications automatiques à la fin (pas de réponse vide, pas d'ID en double, pas d'ID manquant) avant de sauvegarder le fichier final.

CE QU'IL FAUT RETENIR

Chaque notebook a un rôle précis et s'appuie sur le résultat du précédent : 00 prépare l'environnement, 01 comprend les données, 02 les transforme, 03 entraîne le modèle, 04 vérifie s'il a bien appris, 05 produit les réponses finales. Aucune étape ne peut être sautée.

Tous les fichiers générés, et à quoi ils servent

En une phrase : chaque notebook laisse des traces sur Google Drive — voici ce que contient chaque dossier, et pourquoi il existe.

8.1 — Dans le dépôt GitHub : `data/`

Les données de départ, fournies par les organisateurs (jamais modifiées) : `Train.csv` , `Val.csv` , `Test.csv` , `SampleSubmission.csv` (un exemple du format de réponse attendu).

8.2 — Sur Drive : `processed_data/`

Créé par le notebook 02. Contient les données **transformées**, prêtes pour l'entraînement :

- `train_balanced.jsonl` — les 53 300 exemples d'entraînement, rééquilibrés et mis en forme.
- `dev_split.jsonl` — les 1 491 exemples de surveillance en cours d'entraînement.
- `val_formatted.jsonl` — `Val.csv` mis en forme, prêt pour l'évaluation.

DÉFINITION — `.JSONL`

Un fichier `.jsonl` (*JSON Lines*) contient une ligne de données par ligne de fichier, chacune au format JSON (un format texte structuré, lisible par un programme). C'est un format pratique pour stocker beaucoup d'exemples similaires.

8.3 — Sur Drive : `checkpoints/`

Créé par le notebook 03. C'est le dossier le plus important : il contient **ce que le modèle a appris**.

- `gemma-4-e2b-lora-l4-bf16/checkpoint-XXXX/` — des sauvegardes intermédiaires prises pendant l'entraînement (aux pas 6400, 9900, 9996...), qui permettent de reprendre l'entraînement si la session Colab se coupe.
- `gemma-4-e2b-lora-final/` — l'adaptateur LoRA final (~115 Mo), celui qu'on réutilise dans les notebooks 04 et 05. Contient notamment `adapter_model.safetensors` (les poids appris) et `adapter_config.json` (les réglages utilisés, comme `r=16`).

RAPPEL IMPORTANT

Ce dossier ne contient **pas** le modèle Gemma complet — juste le petit "supplément" LoRA. C'est pour ça qu'il faut retélécharger Gemma depuis Hugging Face à chaque fois (question déjà répondue plus tôt dans cette conversation).

8.4 — Sur Drive : `eval_results/`

Créé par le notebook 04. Contient les résultats chiffrés de l'évaluation : `rouge_summary.csv` (scores globaux), `rouge_by_language.csv` (scores par langue), `val_eval_detailed.csv` (chaque question, sa vraie réponse, et les 2 réponses générées — avec et sans fine-tuning).

8.5 — Sur Drive : `submissions/`

Créé par le notebook 05. Contient `submission_gemma4_e2b_finetuned.csv` : les 2 618 réponses finales générées pour `Test.csv`, au format attendu par Zindi (même si on ne soumet pas réellement).

8.6 — Sur Drive : `logs/`

Prévu au départ (Phase 0) pour stocker des journaux d'entraînement détaillés (par exemple avec un outil comme TensorBoard). Dans notre cas, la progression de l'entraînement a été suivie directement dans les sorties affichées par Colab plutôt que via cet outil — ce dossier est donc resté peu utilisé.

CE QU'IL FAUT RETENIR

`processed_data/` = les données prêtes à apprendre. `checkpoints/` = ce que le modèle a appris (l'adaptateur LoRA). `eval_results/` = les notes obtenues. `submissions/` = les réponses finales produites. Chaque dossier correspond à la sortie d'un notebook précis.

Comprendre nos résultats

En une phrase : le fine-tuning a clairement amélioré les réponses du modèle, dans toutes les langues, mais l'écart entre l'anglais et les langues locales persiste.

9.1 — C'est quoi le score ROUGE ?

DÉFINITION

ROUGE est une méthode qui compare automatiquement un texte généré par le modèle à un texte de référence (la vraie réponse), en comptant combien de mots (ou de suites de mots) ils ont en commun. Le score va de 0 (aucun mot en commun) à 1 (texte identique).

On utilise deux variantes : **ROUGE-1** (compare mot par mot, sans tenir compte de l'ordre) et **ROUGE-L** (compare la plus longue suite de mots communs dans le même ordre — plus strict).

9.2 — Nos résultats globaux

Métrique	Sans fine-tuning (zero-shot)	Avec fine-tuning	Amélioration
ROUGE-1	0,1294	0,3667	×2,8
ROUGE-L	0,0794	0,3109	×3,9

9.3 — Nos résultats par langue

Langue	ROUGE-1 zero-shot	ROUGE-1 fine-tuné
Eng_Uga	0,185	0,573
Eng_Ken	0,204	0,495
Eng_Eth	0,070	0,456
Swa_Ken	0,197	0,373
Eng_Gha	0,185	0,362
Aka_Gha	0,095	0,266
Lug_Uga	0,007	0,186
Amh_Eth	0,051	0,156

9.4 — Ce que ça veut dire concrètement

- **Le fine-tuning a aidé sur toutes les langues, sans exception.** C'est la preuve que la méthode fonctionne.
- **Le cas le plus impressionnant : le luganda.** Avant fine-tuning, le modèle ne savait quasiment pas répondre en luganda (score proche de zéro — il répétait le même mot en boucle). Après fine-tuning, son score est multiplié par 27.
- **Mais l'écart entre langues persiste.** L'amharique et le luganda restent les langues où le modèle obtient les scores les plus bas, exactement comme l'EDA (Partie 4.4) l'avait laissé prévoir : ce sont les langues les moins bien connues du tokenizer de Gemma.

EXEMPLE CONCRET D'AMÉLIORATION QUALITATIVE

Sur une question posée en akan, le modèle **sans** fine-tuning a répondu... en français ! Après fine-tuning, la même question obtient une réponse cohérente, en akan, sur le bon sujet.

CE QU'IL FAUT RETENIR

Le fine-tuning multiplie la qualité des réponses par 2,8 à 3,9 selon la mesure utilisée. Le modèle apprend même à "parler" une langue qu'il ne maîtrisait presque pas avant (luganda). La difficulté restante (amharique, luganda, akan un peu en retrait) est cohérente avec ce que l'analyse des données avait annoncé dès le départ.

Les problèmes techniques rencontrés, expliqués simplement

En une phrase : travailler avec un modèle tout juste sorti, ça veut dire rencontrer (et résoudre) des bugs que personne n'a encore documentés.

`google/gemma-4-E2B-it` est un modèle très récent. Résultat : plusieurs outils utilisés (les bibliothèques Python `transformers`, `peft`, `bitsandbytes`) n'étaient pas encore parfaitement à jour avec ses particularités. Voici les principaux problèmes rencontrés, en langage simple.

Problème 1 — "Le modèle est trop gros pour la mémoire" (au moment de préparer l'entraînement)

Une fonction censée préparer le modèle pour l'entraînement essayait de "dé-compresser" des parties énormes du modèle (le dictionnaire de mots, cf. Partie 2.3) d'un coup, ce qui demandait plus de mémoire que disponible. **Solution** : on a remplacé cette préparation par une version plus légère, qui fait juste le strict nécessaire.

Problème 2 — "Certaines parties du modèle veulent aller sur le processeur (CPU) au lieu de la carte graphique (GPU)"

Le système a mal évalué la mémoire disponible et a voulu mettre une partie du modèle sur le CPU (beaucoup plus lent). **Solution** : on a forcé explicitement tout le modèle à rester sur le GPU, en lui prouvant qu'il y a la place.

Problème 3 — "Une pièce du modèle n'est pas reconnue"

Gemma-4 utilise une pièce interne un peu spéciale (une couche avec un "plafonnement" de sécurité) que l'outil LoRA ne savait pas encore reconnaître. **Solution** : on a écrit un petit bout de code qui "déballe" cette pièce spéciale pour révéler la pièce standard qu'il y a dedans, que LoRA sait utiliser.

Problème 4 — "Les deux fichiers de données ne se ressemblent pas assez"

Deux fichiers de données (entraînement et surveillance) n'avaient pas exactement les mêmes colonnes, ce qui a fait planter le chargement. **Solution** : on les a rendus cohérents entre eux.

Problème 5 — "Impossible de reprendre après une coupure"

En changeant de GPU en cours de route (Partie 6.3 précise, T4 → L4), certains réglages internes de sauvegarde n'étaient plus compatibles avec les anciennes sauvegardes. **Solution** : on a créé un nouveau dossier de sauvegarde propre pour le nouveau GPU.

Problème 6 — "Le modèle ne reçoit plus les bonnes informations"

Un mécanisme de nettoyage automatique retirait par erreur les colonnes de texte dont on avait besoin. **Solution** : on a désactivé ce nettoyage automatique.

Problème 7 — "Trop de questions à la fois"

Le calcul de l'erreur du modèle (Partie 1.6, étape 2) demandait plus de mémoire que prévu quand on traitait 8 questions en même temps, à cause du très grand dictionnaire de Gemma. **Solution** : on est passé à 4 questions à la fois (en compensant pour garder le même résultat d'apprentissage global).

CE QU'IL FAUT RETENIR

Aucun de ces problèmes n'est une erreur de conception du projet : ce sont des frictions normales quand on utilise un modèle tout juste publié. Chacun a été diagnostiqué à partir d'un vrai message d'erreur, puis corrigé, un par un — c'est une part importante et normale du travail d'ingénierie en IA.

Glossaire complet (ordre alphabétique)

Adaptateur (LoRA)

Les petites matrices supplémentaires entraînées par LoRA, sauvegardées à part du modèle de base.

Batch (lot)

Un petit paquet d'exemples traités en même temps par le modèle pendant l'entraînement, au lieu d'un seul à la fois — plus rapide et plus stable.

bf16 / fp16

Deux façons de stocker les nombres avec moins de précision que d'habitude, pour aller plus vite et utiliser moins de mémoire. **bf16** est plus stable mais demande un GPU récent (comme le L4) ; **fp16** fonctionne sur des GPU plus anciens (comme le T4).

Checkpoint

Une sauvegarde de l'état du modèle à un instant donné pendant l'entraînement, qui permet de reprendre sans repartir de zéro.

Embedding

Une liste de nombres qui représente le "sens" d'un mot ou d'un morceau de mot pour le modèle.

Epoch (époque)

Un passage complet sur tous les exemples d'entraînement.

EDA

Analyse exploratoire des données : regarder et comprendre les données avant de construire un modèle.

Fine-tuning

Spécialiser un modèle déjà pré-entraîné sur une tâche précise, avec un jeu de données plus petit.

Gradient checkpointing

Une technique qui économise de la mémoire pendant l'entraînement, en recalculant certaines informations au lieu de toutes les garder en mémoire (au prix d'un peu plus de temps de calcul).

GPU

Carte graphique — un composant informatique très efficace pour faire énormément de calculs mathématiques en parallèle, ce qui le rend indispensable pour l'IA.

Learning rate (taux d'apprentissage)

La taille du "pas" à chaque petit ajustement des poids pendant l'entraînement. Trop grand : le modèle apprend n'importe quoi. Trop petit : il apprend trop lentement.

LLM

Grand modèle de langage — un réseau de neurones entraîné sur d'énormes quantités de texte.

Loss (perte)

Un nombre qui mesure à quel point le modèle se trompe. Plus il est bas, mieux le modèle prédit.

LoRA

Technique qui gèle le modèle de base et n'entraîne que de petites matrices supplémentaires.

Machine Learning

Apprentissage automatique : un programme qui apprend à partir de données au lieu de suivre des règles écrites à la main.

Matrice

Un grand tableau de nombres organisé en lignes et colonnes.

Overfitting (sur-apprentissage)

Quand un modèle apprend "par cœur" les exemples d'entraînement au lieu de comprendre le principe général — il devient alors mauvais sur des exemples nouveaux.

PEFT

Parameter-Efficient Fine-Tuning : la famille de techniques (dont LoRA) qui permettent de fine-tuner un modèle en n'entraînant qu'une petite partie de ses poids.

Poids (weight)

Un petit nombre réglable qui détermine l'importance d'une information dans le calcul d'un neurone.

Prompt

Le texte d'instruction/question donné en entrée au modèle.

Quantification

Compresser les nombres d'un modèle (par exemple en 4 bits au lieu de 16 ou 32) pour réduire la mémoire nécessaire.

QLoRA

LoRA appliqué à un modèle de base quantifié (comprimé) — permet d'entraîner de grands modèles sur du matériel limité.

Réseau de neurones

Un programme organisé en couches de petites unités de calcul, qui apprend à partir d'exemples.

ROUGE

Une méthode qui mesure automatiquement la ressemblance entre un texte généré et un texte de référence.

Token

Un petit morceau de texte (mot ou partie de mot) — l'unité de base que le modèle manipule.

Tokenizer

L'outil qui découpe le texte en tokens et les convertit en nombres.

Zero-shot

Utiliser un modèle "tel quel", sans aucun fine-tuning sur la tâche précise — sert de point de comparaison pour mesurer l'apport du fine-tuning.

Pour aller plus loin

En une phrase : vous avez maintenant les bases pour comprendre n'importe quel projet de fine-tuning de LLM — la suite, c'est de la pratique.

Ce que vous savez faire, maintenant

- Expliquer ce qu'est un poids, une matrice, un réseau de neurones, et ce que veut dire "entraîner" un modèle.
- Expliquer comment un LLM transforme du texte en nombres, et génère sa réponse mot par mot.
- Expliquer pourquoi on a choisi le fine-tuning plutôt que d'autres approches, avec des arguments concrets.
- Expliquer LoRA et QLoRA avec vos propres mots, et à quoi servent `r`, `alpha`, la quantification 4-bit.
- Décrire ce que fait chacun des 6 notebooks du projet, et pourquoi dans cet ordre.
- Lire un tableau de résultats ROUGE et l'interpréter.

Trois pistes si vous voulez creuser encore

1. **Relire ce document une deuxième fois**, une semaine après la première lecture — beaucoup de notions ici s'ancrent mieux à la relecture, une fois la structure générale déjà en tête.
2. **Rouvrir un notebook** (par exemple `02_build_dataset.ipynb`) et essayer de retrouver, ligne de code par ligne de code, à quelle étape de ce cours elle correspond.
3. **Reformuler à voix haute**, comme si vous l'expliquiez à quelqu'un qui n'y connaît rien — c'est souvent le meilleur test pour savoir ce qu'on a vraiment compris.

Document généré pour accompagner le projet gemmafro-e2b (Master 2 BI, UCAD). Voir aussi `docs/RAPPORT.md` (rapport de projet détaillé) et `docs/SLIDES.md` (plan de présentation).